

Урок 22. Корутины. Flow. Ошибка, отмена, повтор.

11 сентября 2020

В этом уроке рассматриваем операторы `catch`, `retry`, `retryWhen` и возможность отмены Flow.

Если в Flow возникает ошибка, то мы получим ее там где происходит вызов метода `collect()`. Потому что, как мы знаем, именно в этом методе и запускается вся работа.

Рассмотрим простой пример Flow, в котором происходит ошибка:

```
1 var flow = flow {
2     delay(500)
3     emit("1")
4     delay(500)
5     emit("2")
6
7     val a = 1 / 0
8
9     delay(500)
10    emit("3")
11    delay(500)
12    emit("4")
13 }
```

После отправки второго элемента попробуем выполнить деление на 0.

Получение данных:

```
1 launch {
2     flow.collect {
3         log("collect $it")
4     }
5 }
```

Мы получим первые два элемента, а потом метод `collect` выбросит исключение. Реакция корутины на это будет стандартной. Если ошибка нами не ловится и нет никакого Handler, то ошибка пойдет в системный обработчик, и это приведет к крэшу.

Как вариант, можно обернуть вызов `collect` в `try-catch`.

```
1 launch {
2     try {
3         flow.collect {
4             log("collect $it")
5         }
6     } catch (e: Exception) {
7         log("exception $e")
8     }
9 }
```

Это сработает и ошибка не приведет ни к каким последствиям. А Flow просто завершит работу.

Кроме стандартного `try-catch` варианта нам доступны специальные Flow операторы для работы с ошибками. Давайте рассмотрим их.

catch

Оператор `catch` является аналогом стандартного `try-catch`. Он перехватит ошибку, чтобы она не ушла в `collect`.

```
1 launch {
2     flow
3         .catch { log("catch $it") }
4         .collect {
5             log("collect $it")
6         }
7 }
```

Добавляем оператор `catch`. Он поймает ошибку и выведет ее в лог. Передача данных из Flow при этом прекратится, а метод `collect()` не выбросит исключение.

Оператор `catch` сможет поймать ошибку только из предшествующих ему операторов. Если ошибка возникла в операторе, который в цепочке находится после `catch`, то она не будет поймана этим `catch`.

Рассмотрим на примере:

```

1 | var flow = flow {
2 |     delay(500)
3 |     emit("1")
4 |     delay(500)
5 |     emit("2")
6 |     delay(500)
7 |     emit("a")
8 |     delay(500)
9 |     emit("4")
10| }

```

Flow отправляет строки: 1,2,a,4.

На стороне получателя добавим оператор `map`, который будет преобразовывать данные в цифры, что должно привести к ошибке на третьем элементе:

```

1 | launch {
2 |     flow
3 |         .catch { log("catch $it") }
4 |         .map { it.toInt() }
5 |         .collect {
6 |             log("collect $it")
7 |         }
8 | }

```

При запуске мы в `map` получим ошибку, т.к. строку "a" не получилось преобразовать в цифру. Но `catch` не поймает эту ошибку, потому что `map` в цепочке операторов расположен после `catch`. В итоге метод `collect` выбросит это исключение в наш код.

Чтобы исправить это, нужно просто `catch` поставить после `map`

```

1 | launch {
2 |     flow
3 |         .map { it.toInt() }
4 |         .catch { log("catch $it") }
5 |         .collect {
6 |             log("collect $it")
7 |         }
8 | }

```

Теперь `catch` поймает ошибку из `map`.

Также можно использовать несколько операторов `catch`, если это необходимо:

```

1 | launch {
2 |     flow
3 |         .catch { log("catch $it") }
4 |         .map { it.toInt() }
5 |         .catch { log("catch2 $it") }
6 |         .collect {
7 |             log("collect $it")
8 |         }
9 | }

```

Используем два оператора `catch`. Первый будет ловить ошибки из `flow`, а второй - из `map`.

Также в блоке кода `catch` нам доступен метод `emit()`. Т.е. мы можем отправить какие-то свои данные получателю в случае возникновения ошибки.

retry

Оператор `retry` перезапустит `Flow` в случае ошибки. Как и оператор `catch`, он срабатывает только для тех ошибок, которые произошли в предшествующих ему операторах.

Пример:

```

1 | launch {
2 |     flow
3 |         .retry(2)
4 |         .collect {
5 |             log("collect $it")
6 |         }
7 | }

```

На вход оператор `retry` принимает количество попыток. Если все попытки закончатся ошибкой, то `retry` просто ретранслирует эту ошибку дальше. Поэтому добавляйте `catch` после `retry` если хотите ошибку в итоге поймать.

Если на вход `retry` не передавать количество попыток, то по умолчанию оно будет равно `Long.MAX_VALUE`. Упорству такого `retry` можно только позавидовать.

Кроме количества попыток, в оператор `retry` можно передать блок кода. На вход в этот блок `retry` даст нам пойманную ошибку. А от нас он ждет `Boolean` ответ, который определяет, надо ли пытаться перезапустить `Flow`

```

1 launch {
2     flow
3         .retry(2) {
4             it !is ArithmeticException
5         }
6         .collect {
7             log("collect $it")
8         }
9     }

```

В блоке `retry` мы проверяем тип ошибки. Если это не `ArithmeticException`, то вернется `true` - надо перезапустить `Flow`. А если ошибка - `ArithmeticException`, то вернется `false` - `Flow` не будет перезапущен, а ошибка пойдет дальше, и ее надо будет ловить.

Если вам нужно, чтобы `retry` выждал какую-то паузу прежде чем перезапустить `Flow`, можно использовать `delay`:

```

1 launch {
2     flow
3         .retry(2) {
4             if (it is NetworkErrorException) {
5                 delay(5000)
6                 true
7             }
8             false
9         }
10        .collect {
11            log("collect $it")
12        }
13    }

```

Здесь мы проверяем тип ошибки. Если это `NetworkErrorException`, то ждем 5 сек и возвращаем `true`, чтобы перезапустить `Flow`. В любом другом случае мы сразу возвращаем `false` - `Flow` не будет перезапущен, а ошибка пойдет дальше.

retryWhen

Оператор `retryWhen` работает примерно так же, как и `retry`. Но он не принимает на вход количество попыток. Вместо этого он дает нам в блок кода номер текущей попытки и ошибку. А нам уже надо решить - возвращать `true` или `false`.

```

1 launch {
2     flow
3         .retryWhen { cause, attempt ->
4             cause is NetworkErrorException && attempt < 5
5         }
6         .collect {
7             log("collect $it")
8         }
9     }

```

Если тип ошибки `NetworkErrorException` и количество попыток меньше 5, то возвращаем `true`, чтобы перезапустить `Flow`.

Также в блоке кода `retryWhen` нам доступен метод `emit()`. Т.е. мы можем отправить какие-то свои данные получателю.

Отмена

На стороне получателя мы всегда можем остановить `Flow`, вызвав метод `cancel()`. Но этот метод не относится непосредственно к `Flow`. Это стандартный метод отмены корутины.

```

1 launch {
2     flow
3         .collect {
4             if (it == "3") cancel()
5             log("collect $it")
6         }
7     }

```

Вызывая метод `cancel()` мы отменяем текущую корутину, в которой выполняется `Flow`. А метод `emit` в блоке `flow` всегда проверяет это и выбросит `JobCancellationException` (как стандартная `cancellable suspend` функция).

В итоге работа `Flow` завершится. А метод `collect` выбросит в наш код `JobCancellationException`. Поэтому, если в нашей корутине есть код после метода `collect` и нам надо, чтобы он был выполнен даже после вызова `cancel`, то вызов `collect` надо оборачивать в `try-catch`. И в последующем коде учитывайте, что корутина будет находиться в отмененном состоянии. Т.е. `isActive` будет возвращать `false`, а `cancellable suspend` функции (типа `delay`) будут выбрасывать `JobCancellationException` исключение.

Если `Flow` создан не билдером `flow`, а, например, `asFlow`:

```

1 (1..3).asFlow()

```

то он работает немного по-другому. Его `emit` не будет сам проверять корутину на отмену.

Чтобы научить его это делать, нужно использовать оператор cancellable:

```
1 | (1..3).asFlow().cancellable()
```

